

Kivi: Phonetic Word Memory

"The Words Kivi Keeps" - Sarvam Backend-Focused Full Stack take-home

26/26 eval

67/67 tests

P/R/F1 = 1.0

A word-level memory system for Kivi's dictation pipeline. It learns personal terms (names, products, places) from the corrections a user makes, and applies that memory to fix the same terms later - but only once a correction has been genuinely confirmed twice, never off a single guess.

How it works: three transcript stages

STAGE 1
ASR output

Raw speech-to-text, as heard.

ask aditya to review the sarvam kiwi service

STAGE 2
Formatted output

Sentence-cased + punctuated. Deterministic, no NER/LLM.

Ask aditya to review the sarvam kiwi service.

STAGE 3
Memory-aware output

Personal terms corrected using confirmed memory.

Ask Aaditya to review the sarvam Kivi service.

How a transcript gets corrected (Stage 2 → 3, at request time)

tokenize()	→	retrieve()	→	decide()	→	apply()
splits into words, keeps offsets & casing		finds exact + fuzzy candidate memories (routed by table size - see below)		4 rules pick intervene vs. leave-alone, w/ a reason		substitutes only the decided spans, keeps whitespace/ punctuation exact

The 4 decision rules, in order: **(1)** already correct → leave alone. **(2)** memory not yet active (needs 2 confirmations) → leave alone. **(3)** a small hand-curated list of ambiguous common words (e.g. "apple") needs a nearby supporting cue ("stock", "Inc") before applying. **(4)** conflicting active memories for the same span → highest confidence wins. Every decision returns a human-readable reason + explanation, whether it acted or not.

How a memory gets learned (three independent ways in, one shared core)

Path	Input	Mechanism
Teach one pair	1 (asr, corrected)	Diff formatted vs. corrected
Bulk pairs	many (asr, corrected)	Same diff, looped
Bulk conversations	correct text only, no ASR	NER (free) + optional LLM refine

Confidence: $\min(1.0, \text{evidence_count} / 3)$. A memory only applies once $\text{evidence_count} \geq 2$ AND $\text{confidence} \geq 0.6$ - both cross at exactly the same point, so they can't disagree. One correction is never enough on its own; it could be a typo.

NER + LLM hybrid, verified not assumed

For "corrected text only" input there's no wrong/right pair to diff, so entities must be *extracted*. spaCy NER runs first - free, offline, no API key. An LLM pass refines it only if credentials happen to be configured; a missing key is treated as normal, not an error. Real, measured result on one sentence:

Entity	Found by	Why
Kivi, Bengaluru	NER and LLM	Ordinary place/org patterns spaCy recognizes reliably
Aaditya, Sarvam	LLM only	Invented/unusual spellings - exactly spaCy's verified recall gap

This is why the LLM stays a *refinement*, not a requirement: with only the NER model installed (one-time, free download), this mode already works with zero LLM credentials.

What it can tell apart (26 eval cases, beyond the one worked example)

Needs 2 confirmations before acting	Leaves already-correct text alone
Blocks ambiguous words without supporting context ("apple")	...but allows it once context appears ("Apple" + "stock")
Doesn't over-gate ordinary words not on the ambiguity list ("kiwi")	Handles ALL-CAPS / mixed-case ASR
Handles possessives ("aditya's")	Handles multi-word entities ("yolk sity" → "York City")
Fuzzy-matches small ASR typos within a budget	...but never matches unrelated/very short words
Resolves conflicting evidence toward the most recent	Applies multiple memories correctly in one sentence

Architecture

```
Browser (frontend/, vanilla JS) -> FastAPI (backend/app.py, routes.py)
+-- POST /observe(/bulk)          -> diff.py -> memory_service.py -> SQLite
+-- POST /learn-from-conversations -> ner_extraction.py + entity_extraction.py (LLM, optional)
+-- POST /run                     -> formatter -> tokenizer -> retrieval -> decision -> apply
+-- GET/DELETE /memories, POST /reset
```

Two tables: memory (one row per learned pair, UNIQUE(observed_form, canonical_form)) and evidence (append-only log, source_type = observed_pair or conversation_extraction - every memory's full history is inspectable, not just its current state).

Retrieval performance at scale (added after initial submission)

backend/retrieval.py now routes to one of two matching strategies based on memory-table size, instead of always scanning every row:

Memory rows	Strategy	5-transcript retrieve(), 50k-row test
≤ RETRIEVAL_SMALL_TABLE_THRESHOLD (2,000)	Bulk load once, match in Python - the original design	within ~2x of the original at small sizes
> 2,000	Exact match batched into indexed SQL queries; fuzzy match via a cached symmetric-delete (SymSpell) index instead of a Levenshtein scan against every row	up to 106x faster than always-brute-force at 50,000 rows

Measured in eval/benchmark_retrieval.py, which reproduces the original pre-optimization algorithm for a like-for-like comparison at 100 / 1,000 / 5,000 / 20,000 / 50,000 memory rows. tests/test_retrieval_router.py asserts both strategies return identical candidates on the same input.

Limitations (known and deliberate)

- **Multi-word spans match exactly** - no fuzzy tolerance for spans > 1 token.
- **A taught multi-word entity can fragment** into separate single/multi-word memories, since diffing has no real entity extraction. Mitigated (optional): an LLM grouping pass can merge them when credentials are set.
- **Ambiguous-word stoplist is small and hand-curated**, not a full dictionary - deliberately, to avoid over-gating ordinary product names.
- **No automatic decay** for superseded memories; conflicting evidence competes at retrieval time instead. Manual delete is the “forget” control.
- **Entity-type inference is a coarse heuristic** (nearby cue words), not real NER, in the core diff path.
- **Single-user, SQLite only** - out of scope per the brief’s own “smallest system” framing.

Design decisions, in brief

Decision	Why
Formatter stays deliberately minimal (no NER)	Assignment scope is phonetic memory, not general formatting - the gap is intentional, memory fills it
Deterministic substitution, not LLM prompt-injection	Part Two needs byte-exact, reproducible evaluation
No Alembic	One schema version for this scope; idempotent CREATE TABLE is enough
No mandatory LLM key anywhere in the primary path	Brief requires zero-credential review; NER hybrid extends this to bulk-conversation import too
Retrieval strategy routed by table size, not a single fixed algorithm	An indexed/SymSpell strategy only pays off once memory is large; below that, in-memory scanning is genuinely faster - caught by benchmarking both, not assumed

Full detail, rationale, and verified findings for every claim above: README.md and RUN.md in the repository. Built by me with Claude Code; every number cited here was actually executed and checked in-session, not asserted from memory.